HACKATHON – PARTICIPANT DOCUMENT

AI Banking Copilot for Fraud Detection

on Microsoft Fabric

A challenge focused on bank fraud detection, decision explainability, and analyst assistance via an intelligent copilot.

Objective	Design an AI application capable of detecting, explaining, and prioritizing suspicious transactions, and helping the fraud analyst make decisions.
Data	100% synthetic banking dataset: 2,500 customers, ~435,000 transactions, ~192,000 digital banking events (January → September 2026).
Data platform	Microsoft Fabric (Lakehouse, Notebooks)
Business copilot	Azure OpenAI + interactive interface
Recommended interface	Streamlit
Development assistance	GitHub Copilot

This document replaces the previous version of the brief. It retains the challenge framework and adds everything needed to get started quickly: data description, recommended workflows, feature ideas, copilot design, architecture

Fabric, schedule, evaluation rubric, and starter code.

t

Table of Contents

The essentials on one page

1. Challenge theme and vision
2. Context and problem statement
3. Expected outcomes
4. Technology stack
5. Hackathon data
6. Three recommended paths
7. Feature engineering: where to start
8. Detection and scoring
9. Designing the copilot
10. The analyst interface
11. Reference architecture on Microsoft Fabric
12. Recommended organization and schedule
13. Deliverables
14. Score submission (optional)
15. Evaluation criteria
16. Rules of the game
17. Delivering a successful pitch
18. Final message

Appendix A – Glossary

Appendix B – FAQ

Appendix C – Starter code

The essentials on one page

> **Your mission**
>
> Build a copilot for the fraud analyst at a fictional Tunisian retail bank. It must detect suspicious transactions, explain why, interact with the analyst in natural language, and help them decide
>
> : block, escalate, or approve.

You receive	10 synthetic CSV files: customers, cards, merchants, ~435,000 transactions, digital banking events, fraud labels (January → July), alerts from the current rules engine, data dictionary (README.md).
You deliver	A demonstrable application (alert queue, score, explanation, copilot, decision), a data pipeline on Microsoft Fabric, a GitHub repository, and a 5-minute pitch with a live demo.
Minimum path	transactions_train.csv + fraud_labels_train.csv + customers.csv are enough for a complete solution. The other files are bonuses that enhance detection and explanations.
Starting point	The bank's current rules engine generates 85% false positives and misses 58 % of fraud cases. Your solution must do better, and prove it.
Evaluation	Business relevance, detection quality, quality of explanations and copilot, demo, technological consistency (see § 15).

Three golden rules

1. The copilot speaks from the data. Every claim must be verifiable: amount, date, identifier, comparison with historical data.

2. A transaction is assessed using only its past. Never use information that comes after the transaction being analyzed.

3. Make the transaction → score → explanation → decision path visible in under 30 seconds of the demo.

1. Ingestion	2. Detection	3. Explanation	4. Interaction	5. Decision	6. Improvement
Data in Fabric	Features + model	Factors + Azure OpenAI	Analyst ↔ copilot	Block / escalate / approve	Analyst feedback

The final decision always belongs to the analyst: AI augments decision-making; it does not replace it.

1. Challenge theme and vision

Official theme: AI Banking Copilot for Fraud Detection on Microsoft Fabric.

The challenge lies at the intersection of three dimensions: banking fraud detection, business support through generative AI, and data industrialization on Microsoft Fabric.

The goal is neither a simple scoring engine nor a simple chatbot. The expected result is a banking copilot designed for fraud analysts, capable of interpreting signals, explaining anomalies, and providing practical decision support.

1.1 Intended transformation

- Detect anomalies or weak signals from transactional and behavioral data.
- Calculate a risk score to prioritize alerts.
- Explain the decision or suspicion factors in clear, business-oriented language.
- Enable the analyst to ask simple, targeted, and useful questions.

1.2 Copilot positioning

The copilot does not replace the analyst. It is an assistant that speeds up investigations, facilitates understanding, and improves the quality of decisions. It turns complex data into directly actionable information.

2. Context and problem statement

Banking fraud today is multichannel, fast, organized, and adaptive: online payments, mobile transactions, in-store payments, withdrawals, international transactions, or atypical behavior within seemingly legitimate activity.

Issue	Implications
Financial risk	Direct losses (misappropriated funds, unauthorized transactions), scaling effect in the digital sphere, cost of investigation teams.
Regulatory risk	KYC, AML, monitoring: the decision must be traceable, explainable, and justifiable to the regulator.
Customer trust	Poorly handled fraud, or a card wrongly blocked during a trip, damages the company's image and accelerates customer churn.

2.1 Limitations of the current system

- Static rules: amount thresholds, foreign countries, high frequency. Easy to understand, they poorly capture combinations of weak signals and require extensive maintenance.
- Too many false positives: exceeding just one criterion is enough to trigger an alert. Analysts lose time and become fatigued.
- Poor explanations: the analyst must reconstruct the reasoning from multiple sources.
- Slow cycle: detection → manual review → in-depth analysis → decision. The challenge is to reduce the time between alert and decision without compromising quality or traceability.

What the hackathon data tells us

The file legacy_rule_alerts_train.csv contains the 6,324 alerts raised by the current rules engine between January and July 2026, with the analyst's final decision:

- 85% of these alerts were false positives;
- only 42% of fraud cases triggered an alert;
- example: a customer who went to Istanbul for ten days in July generated 21 alerts alone, all unjustified.

This is your benchmark: show how many unnecessary alerts your solution avoids and how many additional fraud cases it detects.

3. Expected results

The final solution must demonstrate four complementary capabilities:

Capability	What the solution must do	What the jury wants to see
Detect	Identify suspicious transactions and assign them a risk score.	A prioritized alert queue, a credible and validated score, and a comparison with the rules engine.
Understand	Explain the factors that led to the score.	Quantified factors compared with the customer's history, not generalities.
Interact	Engage with the analyst to speed up their analysis.	Live answers to open-ended questions, grounded in the data.
Decide	Help the analyst make a decision and keep a record.	A recommendation (block / escalate / approve), a recorded decision, and feedback.

Example of the expected explanation: "This transaction is suspicious because its amount is 5 times higher than the customer's usual spending and it took place in a country where they have never made an in-person payment." The format may vary, but the explanation must be clear, quantified, and decision-oriented.

4. Technology stack

Component	Role in the solution	Status
Microsoft Fabric	Ingestion, storage (Lakehouse), preparation, and features (Notebooks), scoring.	Expected
GitHub + GitHub Copilot	Code versioning and accelerated development (code, tests, interface, documentation).	Expected
Azure OpenAI	Generating explanations and interacting with the analyst (function calling recommended).	Recommended
Streamlit	Analyst interface: alerts, transaction details, customer view, copilot, decision.	Recommended
Python (pandas, PySpark)	Data preparation and feature engineering.	Recommended
scikit-learn, LightGBM, XGBoost, SHAP	Scoring and explainability model.	Optional

You are free to choose your implementation, but the solution must demonstrate coherent use of Fabric as a data platform and GitHub Copilot as a delivery accelerator.

Constraints

- No simple static dashboard.

- No model without a user interface.
- No generic chatbot disconnected from the fraud case data.

5. Hackathon data

5.1 Context

The data simulates the activity of a fictitious Tunisian retail bank from January 1 to September 30, 2026: 2,500 customers (students, individuals, premium, professionals, retirees) across 18 cities, with their domestic or international cards, in-store and online payments, withdrawals, transfers, and use of the mobile app.

- 100% synthetic data: no real customers; all identifiers and merchant names are fictitious.
- Amounts in Tunisian dinars (TND), with 3 decimal places. Timestamps in Tunis time.
- Fraud rate of about 0.65%: the classes are highly imbalanced.
- The complete data dictionary (all columns and codes) is in README.md.

5.2 Files

File	Content	Rows	Priority
transactions_train.csv	Transactions from January to July 2026	333 059	Essential
fraud_labels_train.csv	Fraud label (0/1), fraud type, reporting date of reporting	333 059	Essential
customers.csv	Customer profile: age, segment, income, city, KYC, digital usage	2 500	Essential
transactions_test.csv	Transactions from August to September 2026, without label (to be scored)	101 730	Essential
merchants.csv	Merchants, e-commerce sites, and ATMs	2 621	Recommended
cards.csv	Card agreements: type, international or not, limits, contactless	2 940	Recommended
legacy_rule_alerts_train.csv	Current rules engine alerts + decision by the analyst	6 324	Recommended
digital_events.csv	Logins, failures, new device, addition of beneficiary, password, phone number	192 401	Bonus (key for the account takeover)
sample_submission.csv	Score submission format	101 730	If ranking
README.md	Complete data dictionary	–	Read first

5.3 How the files are linked

Link	Join key
Transaction ↔ label / rules engine alert	transaction_id
Transaction ↔ customer ↔ card	customer_id, card_id
Transaction ↔ merchant / ATM / website	merchant_id
Mobile or web transaction ↔ digital events from the same session	session_id
Devices, beneficiaries (shared across multiple transactions or customers)	device_id, beneficiary_id

5.4 Time-based split

Period	Data	Recommended use
January → May 2026	Labeled transactions	Model training
June → July 2026	Labeled transactions	Validation (honest performance measurement)
August → September 2026	Unlabeled transactions	Final scoring, demo alert queue, submission optional

For the demo, treat the August and September transactions as the analyst's daily feed: you don't know the answer, just like in real life. Fraud labeled from January to July is used to illustrate and calibrate your explanations.

5.5 Codes to know

Column	Values	What to know
channel	POS, ATM, ONLINE, MOBILE, WEB	POS = in-store payment, ATM = cash machine. MOBILE and WEB = online banking (with session_id).
entry_mode	CHIP, CONTACTLESS, MAGSTRIPE, ECOM_3DS, ECOM_NO_3DS, APP_AUTH	The magnetic stripe (MAGSTRIPE) is easy to copy. Contactless payment does not require a PIN below 200 TND.
transaction_type	PURCHASE, WITHDRAWAL, TRANSFER, BILL_PAYMENT, TOP_UP	Transfers have no card or merchant, but have a beneficiary_id.
status / decline_reason	APPROVED / DECLINED + reason	Reasons: insufficient balance, limit exceeded, incorrect PIN, invalid CVV, invalid expiration date, card not enabled for international use, blocked card. Declined **attempts may be fraud.**
event_type	LOGIN, ADD_DEVICE, ADD_BENEFICIARY, PASSWORD_RESET, CHANGE_PHONE_NUMBER	With status (SUCCESS / FAILURE), channel (including BRANCH and CALL_CENTER) and auth_method.

Empty cells are normal: no card for a transfer, no device for an in-store payment, no coordinates for an online purchase. Details are in README.md.

5.6 Fraud typologies

Each fraudulent transaction in the training period has a typology (fraud_scenario). These typologies are an excellent basis for your explanations and for measuring what your model detects well or poorly.

Typology	What happens	Where to look
CARD_TESTING	Stolen card data are Bursts of tiny amounts, repeated declines, then used for large purchases.	tested with small online payments, unusual device or IP, categories easily resold.
ACCOUNT_TAKEOVER	A fraudster takes control of online banking access and empties the account through transfers.	digital_events just before the transfer: failed logins, new device, number change, new beneficiary. Amount compared with usual patterns.

.(C

Typology	What happens	What to look for
CLONED_CARD_ABROAD	A copy of the card is used physically abroad.	Country never visited for in-person payment, entry mode, simultaneous activity by the real customer in Tunisia.
ATM_SKIMMING	The card data have been copied at an ATM. The card is then emptied through withdrawals.	Closely spaced withdrawals, unusual city, times, limits reached.
STOLEN_CONTACTLESS_CARD	A stolen physical card is used without a PIN.	Successive contactless payments just below the limit, merchants unknown to the customer.
LOW-KEY_ONLINE_FRAUD	Fraud spread over several days, with modest amounts.	Combination of weak signals: devices, IPs, categories, frequency. The most **difficult.**

> **Clue: fraud is sometimes organized**
>
> Some fraud cases are not isolated: several victims share a common point (a merchant, an ATM, a website, a device) before being defrauded. A “graph-based” analysis of these common points is a real asset, both for detection and for the demo.

5.7 The pitfalls: legitimate behavior that looks like fraud

Reducing false positives is central to the task. Your copilot must be able to explain why an alert is not fraud.

Legitimate situation	Why it looks like fraud	What helps determine the truth
Travel abroad (vacation, Umrah, business)	New country, high amounts, magnetic stripe on older terminals.	Ticket or hotel booked in advance, more physical activity in Tunisia during the stay, IP address of the country visited.
Summer vacation in another city	Unusual city, cash withdrawals away from home.	Consistent pattern over several days, vacation shops.
Large one-off purchase (furniture, electronics, jewelry)	Amount over 3,000 TND.	Local merchant, chip card, during the daytime, no other indicators.
New phone	New device, sometimes followed by a transfer.	OTP enrollment, no failed login, Tunisian IP.
Forgotten password, number change at the branch	Sensitive events.	Branch channel, no unusual transfer afterward.
VPN, shared computer (office, internet café)	Foreign IP, device used by multiple customers.	Stable customer history, usual amounts.
Ramadan evenings, purchases of Eid	Late transactions, increase in spending.	Calendar (§ 5.8).

5.8 The 2026 calendar to know

Period (indicative dates)	Effect on the data
February 18 → March 19: Ramadan	Late-night activity (until 1 a.m.), increase in grocery shopping, Umrah.
March 10 → 19: end of Ramadan	Clothing purchases for Eid.

Period (indicative dates)	Effect on the data
March 20–21: Eid al-Fitr / May 27–28: Eid al-Adha	Reduced activity.
May 18 → 26: before Eid al-Adha	Large cash withdrawals (sheep purchases).
From the 25th to the 3rd of each month	Payday: more transactions.
July → August	Travel and summer holidays (abroad and in Tunisia).
September 1 → 20	Back-to-school period: education, clothing.
Weekend	More activity on Saturday, less on Sunday.

6. Three recommended tracks

Choose your level of ambition based on your team and the time available. Each level is a complete, demonstrable solution. Start with Bronze, then move up.

	Bronze: solid	Silver: convincing	Gold: expert
Data	transactions + labels + customers +	cards, merchants, alerts from the rules engine	+ digital_events, analysis of commonalities between victims
Features	8 to 10 features behavioral (amount vs. habits, novelty, velocity, time)	20 to 30 features: geography, channel, limits, profile	+ digital and graph features (merchant, ATM, device in common)
Detection	Heuristic score or simple model (logistic regression, tree)	Gradient boosting, temporal validation, numerical comparison against the rules engine	+ SHAP contributions per transaction, threshold chosen based on analysts' capacity
Copilot	Top factors injected into the prompt, questions about a transaction	Function calling: customer history, comparison with habits, alert queue	Multi-tool: link cases to one another, draft the investigation summary, explain false positives
Interface	Alert queue, transaction details, chat	+ 360° customer view (timeline, map), decision	+ graph of commonalities, decision log and feedback

> **A perfectly demonstrated Bronze is better than an unfinished Gold. Connect the copilot to a simple score as early as possible, then improve the score.**

7. Feature engineering: where to start

Raw data is not enough: a transaction is suspicious only in relation to the habits of the customer, card, or device. Each feature should be expressible as an explanatory sentence: that is what will make your copilot convincing.

> **Golden rule: past data only**
>
> A feature calculated for a transaction must use only previous transactions and events. Calculating "the average of the customer's transaction amounts" over the entire period is equivalent to looking into the future: the score looks excellent in testing, then collapses in reality. The jury will check the code of the finalist teams.

```
tx = tx.sort_values(["customer_id", "timestamp"])
g = tx.groupby("customer_id")["amount_tnd"]
# average amounts BEFORE the transaction (shift(1) excludes the current transaction)
tx["avg_amount_before"] = g.transform(lambda s: s.shift(1).expanding().mean())
tx["amount_ratio"] = tx["amount_tnd"] / tx["avg_amount_before"]
# customer's first visit to this country
tx["is_new_country"] = ~tx.duplicated(["customer_id", "country"])
```

7.1 Idea catalog

Category	Feature examples	Associated explanatory statement
Amount	Ratio to the customer's historical average or median, customer, z-score, amount / monthly income, amount / card limit	"Amount 5 times higher than the customer's usual average (151 TND)."
Timing	Time, night, day of the week, payday, Ramadan / Eid, time elapsed since the previous transaction	"Transaction at 3 a.m. although the customer never transacts at night."
Velocity	Number and sum of transactions over 10 min, 1 h, 24 h (by card, customer, device), recent declines	"7 attempts in 25 minutes, 4 of them declined."
Novelty	New country, city, merchant, category, device, beneficiary, IP country	"First in-person payment outside Tunisia in the customer's entire history."
Geography	Distance from home, travel speed since the previous in-person transaction	"Payment in Shenzhen 2 hours after a withdrawal in Kairouan: impossible travel."
Channel and security	Entry mode (magnetic stripe, contactless, without 3DS), decline reason, non-international card	"Magnetic stripe used even though the card is chip card."
Digital banking	Failed logins in 24 h, recently enrolled device, recently, password reset, phone number changed, beneficiary added just beforehand	"Beneficiary added 4 minutes before the transfer, from a device enrolled that same day."
Profile	Age, segment, income, tenure, KYC level, card type	"Retired customer with a domestic card: an online purchase abroad is unusual."
Network (graph)	Merchants or ATMs shared by cards already defrauded, devices shared between customers	"12 other cards used at this ATM were defrauded within the last 3 weeks."

Tip for graph features: count only fraud cases already reported on the transaction date (fraud_reported_date earlier than timestamp). This is the only legitimate use of this column as a feature.

In Fabric, rolling windows can be calculated efficiently using PySpark Window functions (example in Appendix C).

8. Detection and scoring

8.1 Start with a heuristic score

In one hour, a points-based score (new country +3, amount more than 3 times the average +2, burst of transactions +3…) gives you an initial alert queue, a benchmark, and a backup plan. It is also very easy to explain.

8.2 Move to supervised learning

- Logistic regression: transparent, with directly interpretable coefficients.
- Gradient boosting (HistGradientBoosting, LightGBM, XGBoost): generally the best-performing approach for this type of tabular data.
- Handle class imbalance with class_weight, scale_pos_weight, or undersampling of legitimate transactions.

- A hybrid approach (model + a few explicit business rules) is perfectly valid if justified.

8.3 Validate over time, never at random

Train on January → May and validate on June → July. A random split mixes transactions from the same fraud episode between training and validation, greatly overstating performance.

8.4 The right metrics

Metric	Why
Accuracy	Avoid: a model that detects nothing already has 99.3% accuracy.
PR-AUC (average precision)	Primary metric, suitable for highly imbalanced classes.
Precision and recall in the top 1 %	Analysts can only handle a limited volume of alerts (~1,000 over two months). How many of these are genuine fraud? What share of fraud is covered?
Comparison with the rules engine	Precision of ~15% and recall of ~42% over the training period: your baseline.
Recall by fraud type	Shows where your model is strong and where it is weaker (honesty appreciated).

8.5 Make the score explainable

For each flagged transaction, calculate and store its 3 to 5 main factors in a table (for example, gold_alert_explanations): factor name, observed value, usual value, contribution. Possible sources: SHAP values, logistic regression contributions, or triggered rules. The copilot will read this to explain, rather than make things up.

8.6 Pitfalls to avoid

Pitfall	Why it's a problem	Fix
Features calculated over the entire period	Future information leakage, artificially inflated score	Past windows, shift(1), expanding()
fraud_scenario or fraud_reported_date as a feature direct	These are labels, unknown at the time of the transaction	Analysis and target only (see the graph trick in § 7.1)
Random train / validation split	Performance overestimation	Temporal split
Optimizing accuracy	Imbalanced classes	PR-AUC, top 1%
Using the order of numeric identifiers	No business meaning	Do not use IDs as features numeric
Letting the LLM calculate the score	Not reproducible, risk of hallucination. The model calculates, the LLM explains	
Sending thousands of rows in the prompt	Cost, latency, context limit	Tools that return aggregates

9. Designing the copilot

9.1 The principle: ground every answer in the data

The score is calculated by your model. The LLM only retrieves facts using tools, then translates them into a business explanation. It must not state anything that its tools have not returned.

Question	**Azure OpenAI**	**Tool**	**Result**	**Answer**
from the analyst	selects a tool	queries the Gold tables	quantified facts (JSON)	explained and sourced

9.2 Recommended tools (functions)

Tool	What it returns
get_alert_queue(date, n)	The riskiest transactions of a day, with score and main factors.
get_transaction(transaction_id)	Transaction details, its score, and risk factors.
get_customer_profile(customer_id)	Profile and habits: average amount, countries, cities, times, devices, beneficiaries.
compare_to_habits(transaction_id)	Quantified differences between the transaction and the customer's history.
get_customer_history(customer_id, days)	The customer's recent transactions and events.
get_digital_events(customer_id, around)	Logins, devices, beneficiaries, and phone number changes around a transaction.
find_related_cases(transaction_id)	Other cards or customers sharing the same merchant, ATM, device, or beneficiary.
record_decision(transaction_id, decision, comment)	Records the analyst's decision (block / escalate / approve) and their comment.

9.3 Suggested system prompt

You are the Bank's fraud analyst copilot. Respond in French, concisely, factually, and in a structured way. Rules:

- State only what the tools have returned. Cite amounts, dates, and identifiers.
- If information is missing, say so and suggest what to verify.
- Never calculate a score yourself: use the one provided by the model.
- Distinguish facts (data) from your interpretation.
- If the transaction seems legitimate, also explain why.
- End with a recommendation: BLOCK, ESCALATE, or APPROVE, with a confidence level. The final decision belongs to the analyst.

9.4 Recommended response format

1. Verdict	Risk level and model score.
2. Main factors	Three at most, each quantified and compared with the customer's habits.
3. Context	Customer profile, recent activity, relevant digital events.
4. Reassuring factors	What mitigates the risk (planned trip, known device…).
5. Recommendation	BLOCK / ESCALATE / APPROVE and confidence level.

9.5 Example 1: a fraud (real training dataset data)

Analyst: “Why is transaction TX00267400 suspicious?”

High risk (model score: 0.94, illustrative value).

Main factors

- In-person payment in Shenzhen (China): all 155 of the customer’s previous in-person payments and withdrawals took place in Tunisia.
- Amount of 783,611 TND, or 5.2 times their average (151 TND), exceeding 95% of their past transactions.
- 13 minutes earlier, an 800 TND withdrawal at a Shenzhen ATM using the magnetic stripe, typical of a cloned card.

Context: 37-year-old customer, Retail segment, residing in Kairouan; last transaction in Tunisia the previous day at 17:01; no flight ticket or hotel booked.

Recommendation: BLOCK card CARD001261 and contact the customer (high confidence).

Update at 15:55: the same card is used at an ATM in Kairouan. The customer cannot be in both places: cloning is confirmed.

9.6 Example 2: a false positive avoided

Analyst: “The rules engine raised an alert on TX00283105 (3,086 TND in Istanbul). Should we block it?”

Low risk: the alert is very likely unwarranted.

Reassuring indicators

- Flight tickets purchased online on June 19 and 24.
- No in-person transactions in Tunisia since the evening of June 30: the customer is traveling, and their payments in Istanbul have followed a consistent sequence since July 1.
- Chip payment, not magnetic stripe.

Point to note: unusual amount (about 2 times their 95th percentile) at a jewelry store, but consistent with a Business customer (income of 4,893 TND).

Recommendation: APPROVE, possibly with a notification to the customer (medium to high confidence). Note: this trip triggered 21 alerts from the current rules engine, all false positives.

9.7 Questions the jury may ask live

- “Why is this transaction suspicious?” / “What are the main risk factors?”
- “Show me yesterday’s 10 riskiest transactions.”
- “Does this transaction resemble the customer’s usual behavior?”
- “Summarize why this alert should be prioritized.”
- “What happened in the app just before this transfer?”
- “Have other customers used the same merchant or ATM?”
- “Why is this alert not fraud?”
- “Write the investigation summary for the case file.”
- “How many unnecessary alerts do you avoid compared with the current rules engine?”

The number of supported questions matters less than the quality of the answers: grounded in the data, useful, and directly actionable.

9.8 Safeguards

- Low temperature (0 to 0.2) for stable, factual answers.
- Proper handling of unknown cases (nonexistent ID, off-topic question).

- Log questions, tools called, and responses: traceability is a regulatory requirement.

- The decision remains human: the copilot recommends, the analyst decides.
- Never commit Azure OpenAI keys to GitHub: use environment variables or an ignored .env file.

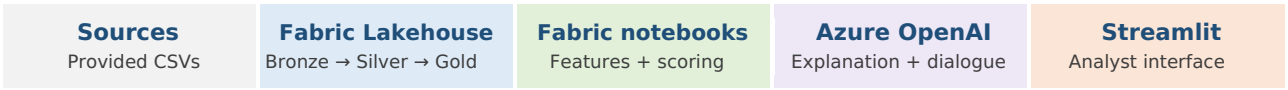
10. The analyst interface

Screen	Content	Tip
Alert queue	Transactions ranked by score, filters (date, channel, type, risk level)	Color-coded by risk level, number of alerts today
Transaction details	Details, score, 3 main factors, decision buttons	This is the highlight of the demo: everything must be visible without scrolling
360° customer view	Profile, habits, transaction timeline and digital events, map of locations	st.map with latitude and longitude, highlighting the flagged transaction
Copilot	Chat area, suggested questions as buttons	Automatically pass the open transaction as context
Decision and feedback	Block / escalate / approve + comment, decision log	Illustrates the continuous improvement loop
Dashboard (optional)	Alert volume, comparison with the rules engine	One chart is enough

For a smooth demo

- Precompute scores and explanations in Fabric: never train the model in the application.
- Load Parquet files or a filtered Gold table, and use st.cache_data.
- Prepare 3 demo cases: obvious fraud, subtle fraud, and an avoided false positive.
- Have a backup video recording ready in case of network issues.

11. Reference architecture on Microsoft Fabric



Across all layers: GitHub + GitHub Copilot for code (notebooks, application, prompts, tests).

Layer	Where	Suggested content
Bronze	Lakehouse (Files + Delta tables)	The CSVs loaded as-is: bronze_transactions_train, , bronze_customers...
Silver	Delta tables	Typed, cleaned, and joined data: silver_transactions (train + test + customer + card + merchant).
Gold	Delta tables	gold_features, gold_scores (score + factors), gold_customer_profile, gold_alert_queue.
Scoring	Fabric notebook (PySpark / pandas, September; MLflow tracking possible)	Training, temporal validation, scoring from August to
Application	Streamlit + Azure OpenAI	Reads Gold tables and calls the LLM using tools.

Connect Streamlit to Fabric data

- Option A: query the Gold tables via the Lakehouse SQL analytics endpoint, using Microsoft Entra authentication.
- Option B (backup plan): export the Gold tables to Parquet from Fabric and read them locally. Simpler if access is difficult, provided the upstream Fabric pipeline is clearly shown.

Why Fabric is central

- It brings ingestion, storage, transformation, and use together in a unified environment.
- It makes the project more production-ready than a local script.
- It strengthens the solution's consistency within the Microsoft ecosystem.

12. Suggested organization and schedule

Phase	Time share Phase-end milestone
1. Scoping and ingestion	10 % Data loaded into Fabric (Bronze), roles assigned, workflow selected.
2. Exploration and features	20 % Feature table (Gold) and 2 or 3 business findings to present.
3. Scoring	20 % Score validated on June → July, compared with the rules engine.
4. Explanation and copilot	25 % The copilot answers the first 4 questions in § 9.7.
5. Interface and integration	15 % Functional end-to-end demo.
6. Pitch and rehearsal	10 % Timed pitch, demo cases finalized, backup video.

Suggested roles (can be combined)

Role	Responsibilities
Data engineer	Fabric Lakehouse, Bronze / Silver / Gold tables, export for the application.
Data scientist	Features, model, validation, explanatory factors.
AI engineer	Copilot tools, system prompt, safeguards, question testing.
Front-end developer / UX	Streamlit application, demo workflow.
Product owner / pitch	Business vision, demo case, storytelling, timing.

> Tip: connect the copilot and interface to an initial heuristic score as soon as possible, then replace it with the model. A team that integrates everything at the last minute is taking a big risk.

13. Deliverables

Deliverable	Expected content
Functional application	Queue of alerts or transactions, score or explicit prioritization logic, simple navigation that can be demonstrated in a few minutes.
Copilot (mandatory)	Dialogue area, open-ended or semi-guided questions, contextualized answers and clear explanations.
Demonstration	Real case (from the data) or simulated, live interaction during the pitch, clear justification for the suspicion.

Deliverable	Expected content
Technical presentation	Architecture, Fabric's role in the pipeline, choice of model or logic for scoring, validation performance.
Use of GitHub Copilot	Concrete examples: generated components, tests, refactoring, time saved.
GitHub repository	Code, exported Fabric notebooks, requirements.txt, README explaining how to run the solution.
Score file (optional)	See § 14.

14. Score submission (optional)

For an additional technical ranking, submit a CSV file containing one row per transaction in transactions_test.csv:

```
transaction_id,fraud_score,is_alert
TX00333060,0.0213,0
TX00333061,0.9731,1
```

- fraud_score (required): continuous score; higher = more suspicious.
- is_alert (optional): your alert decision (0/1) based on your own threshold.

The jury will calculate PR-AUC (the primary metric) and precision / recall in the top 1%. This ranking is a bonus: it does not replace the rubric in § 15. The code of the highest-ranked teams will be reviewed (to check for future information leakage).

15. Evaluation criteria

Criterion	What the jury looks for	Indicative weighting
Business relevance	The solution meets the analyst's real needs: prioritization, decision-making, and reduction of false positives.	20%
Detection quality	Approach, temporal validation, comparison with the rules engine, and absence of information leakage.	20%
Explanation and copilot	Data-grounded, quantified, useful answers, including for legitimate cases.	25%
Experience and demo	Smoothness, clarity, storytelling, live interaction.	15%
Technological coherence	Clear roles for Fabric, Azure OpenAI, and GitHub Copilot, and a clear architecture.	15%
Presentation	Quality of the pitch and answers to the jury's questions.	5%

What is not acceptable

- A dashboard with no intelligence or interaction.
- Scoring without explanation.
- Generic chatbot answers unrelated to the case data.
- A project with no demonstrable interface.

16. Rules of the game

1. The provided data is synthetic and sufficient. If you use public external data, cite it.

2. No information subsequent to a transaction may be used to score it (see § 7 and § 8.6).
3. Test set labels are not disclosed: validation is performed on June → July.
4. AI tools are permitted and encouraged, starting with GitHub Copilot. Indicate how you used them.

5. No keys, passwords, or secrets in the GitHub repository.
6. The jury may request to review any team's code, notebooks, and Git history.

17. Delivering a successful pitch

Suggested format: 5 minutes, live demo at the center.

Time	Content
0:00 – 0:45	The problem, quantified: 85% false positives, 58% of fraud missed by current rules. current rules.
0:45 – 3:00	Live demo: an alert → the transaction record → a question to the copilot → the decision. Then a false positive avoided.
3:00 – 4:00	Under the hood: Fabric pipeline, model, validation results compared with the rules engine.
4:00 – 4:30	How GitHub Copilot saved you time.
4:30 – 5:00	Acknowledged limitations and next steps.

> **The most compelling flow remains: a transaction → a score → an explanation → a decision. The jury should be able to see it in a few seconds.**

18. Final message

This challenge invites you to design more than a model: an intelligent banking assistant that genuinely helps fraud analysts detect, understand, and make decisions. By combining Microsoft Fabric for the data platform, Azure OpenAI for explanations, and GitHub Copilot to accelerate development, you can deliver a modern, credible demonstration with significant business value.

The best proposal will be the one that transforms complex data into actionable decisions, with a simple, clear, and compelling experience. In other words: less noise, more explanation, more value for the analyst.

Good luck at the hackathon!

Appendix A – Glossary

Term	Definition
POS terminal	Electronic payment terminal: card payment in a store.
ATM	Automated teller machine.
3-D Secure (3DS)	Strong cardholder authentication during an online payment (code received by SMS or approval in the app).
Magnetic stripe	Older card-reading technology, easy to copy, unlike the chip.
MCC	Merchant Category Code: code that describes the merchant's business.
OTP	One-Time Password: single-use code, often sent by SMS.
SIM swap	The fraudster transfers the victim's phone number to their own SIM card to receive their OTP codes.
Skimming	Copying card data using a device placed on an ATM or POS terminal.
Card testing	Testing stolen card numbers with small payments before using them for large amounts.
Account takeover (ATO)	A fraudster taking control of an online banking account.
False positive	A legitimate transaction incorrectly flagged as suspicious.
PR-AUC	Area under the precision-recall curve: measures the quality of classification across imbalanced classes.
Information leakage (leakage)	Use during training of information that would not be available at the time of prediction.
Lakehouse	Fabric storage combining files and Delta tables queryable in SQL and Spark.
Bronze / Silver / Gold	Data layers: raw, cleaned and joined, ready for business use.
Function calling	The LLM's ability to call functions in your code (your tools) to obtain data.

Appendix B – FAQ

Do we have to use all the files?

No. Transactions, labels, and customers are sufficient for a complete solution. The other files are optional extras.

Is the data real?

No, it is entirely synthetic. It reproduces realistic behaviors and types of fraud.

Is the fraud rate realistic?

It is intentionally higher than in reality (about 0.65% versus about 0.1%) to give you enough examples.

Can we use the fraud_scenario column?

Yes, as a target (multi-class classification) or to analyze your results, but never as a feature.

How do we evaluate our model without the test labels?

Validate on June → July. The August and September labels are reserved for the judges.

Can we use an LLM other than Azure OpenAI?

Azure OpenAI is recommended. If you use another one, justify your choice.

Can we use Power BI?

Yes, as a supplement, but it replaces neither the interactive interface nor the copilot.

What should you do if Fabric or Azure OpenAI is unavailable?

Notify the organizers immediately. In the meantime, work locally (pandas, Parquet) and switch over as soon as possible.

Appendix C – Starter Code

C.1 Load files into the Lakehouse (Fabric notebook, PySpark)

First place the CSV files in the Lakehouse's Files/hackathon/.

```
files = ["customers", "cards", "merchants", "transactions_train", "transactions_test",
         "fraud_labels_train", "legacy_rule_alerts_train", "digital_events"]
for f in files:
    df = (spark.read.option("header", True).option("inferSchema", True)
          .csv(f"Files/hackathon/{f}.csv"))
    df.write.mode("overwrite").format("delta").saveAsTable(f"bronze_{f}")
```

C.2 A velocity feature (PySpark, past data only)

```
from pyspark.sql import functions as F, Window
tx = spark.table("bronze_transactions_train") \
    .withColumn("ts", F.col("timestamp").cast("timestamp").cast("long"))
# transactions on the same card in the previous hour (current transaction excluded)
w1h = Window.partitionBy("card_id").orderBy("ts").rangeBetween(-3600, -1)
tx = tx.withColumn("card_tx_last_1h", F.count("*").over(w1h))
```

C.3 Call Azure OpenAI with a tool (Python)

```
import os, json
from openai import AzureOpenAI
client = AzureOpenAI(api_key=os.environ["AZURE_OPENAI_KEY"],
                     azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
                     api_version="2024-10-21")
tools = [{"type": "function", "function": {
    "name": "get_transaction",
    "description": "Details, score, and risk factors for a transaction",
    "parameters": {"type": "object",
                   "properties": {"transaction_id": {"type": "string"}},
                   "required": ["transaction_id"]}}}]
messages = [{"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": "Why is TX00267400 suspicious?"}]
resp = client.chat.completions.create(model="<deployment-name>", messages=messages,
                                      tools=tools, temperature=0.1)
# if resp requests a tool: execute the function, add its result to messages,
# then call the API again to get the final response
```

C.4 Streamlit interface skeleton

```
import pandas as pd
import streamlit as st
st.set_page_config(page_title="Fraud Copilot", layout="wide")

@st.cache_data
def load_alerts():
    return pd.read_parquet("gold_alert_queue.parquet")

alerts = load_alerts()
tx_id = st.sidebar.selectbox("Alert", alerts["transaction_id"])
st.dataframe(alerts[alerts.transaction_id == tx_id])

history = st.session_state.setdefault("history", [])
for m in history:
    st.chat_message(m["role"]).write(m["content"])
if question := st.chat_input("Ask your question about this alert..."):
    answer = ask_copilot(question, context_tx=tx_id)          # your Azure OpenAI function
    history += [{"role": "user", "content": question},
                {"role": "assistant", "content": answer}]
    st.rerun()
```

End of document.